

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 15: Randomized Algorithms

Foreign Trade University

Contents

1	Introduction: Flipping Coins to Compute	1
1.1	Two species: Las Vegas and Monte Carlo	1
2	Probability Review	2
3	Contention Resolution: Breaking Symmetry	3
4	Randomized QuickSort	4
4.1	The algorithm	4
4.2	Expected number of comparisons	4
5	Randomized Quickselect	5
6	Karger’s Randomized Global Min-Cut	6
6.1	The problem and the algorithm	6
6.2	Success probability	6
6.3	Amplification	6
7	Reservoir Sampling	7
8	Hashing and Number-Theoretic Randomized Algorithms	8
8.1	Universal hashing	8
8.2	Monte Carlo estimation of π	8
8.3	Miller–Rabin primality testing	8
9	Summary and Course Wrap-Up	8
9.1	This week	8
9.2	Looking back over the course	9

1 Introduction: Flipping Coins to Compute

Every algorithm we have studied so far has been **deterministic**: given the same input, it always performs the same steps and produces the same answer. This week (following Kleinberg & Tardos, Ch. 13) we add a new and surprisingly powerful ingredient – **randomness**. A *randomized algorithm* is allowed to flip coins (call a random number generator) during its execution, and its behaviour on a fixed input may differ from run to run.

Why would we want this? Three recurring reasons:

- **Simplicity and speed.** Randomized QuickSort and randomized Quickselect are shorter and, on average, as fast as or faster than their best deterministic counterparts, while sidestepping worst-case adversarial inputs.
- **Breaking symmetry.** When many identical processes must make a decentralized decision (contention resolution), randomness lets each break the tie independently – something deterministic identical processes cannot do.
- **Solving problems we don’t know how to solve deterministically (efficiently).** Karger’s global min-cut algorithm and the Miller–Rabin primality test are strikingly simple randomized algorithms for problems whose deterministic solutions are far more involved.

The price is that we must reason about *probability*: an analysis no longer ends with “the answer is correct in time $T(n)$,” but with statements like “the *expected* running time is $O(n \log n)$ ” or “the answer is correct with probability at least $1 - \delta$.” The mathematics is elementary – expectation, linearity of expectation, indicator variables, and the union bound – but using it fluently is the real skill this week develops.

1.1 Two species: Las Vegas and Monte Carlo

Definition 1.1 (Las Vegas algorithm). *A Las Vegas algorithm is **always correct**, but its running time is a random variable. We analyze its expected running time. (Randomized QuickSort and Quickselect are Las Vegas algorithms.)*

Definition 1.2 (Monte Carlo algorithm). *A Monte Carlo algorithm runs in a fixed (or bounded) time, but may be **wrong with some bounded probability**. We analyze its success probability, and usually amplify it by repetition. (Karger’s min-cut and Miller–Rabin are Monte Carlo algorithms.)*

A useful slogan: *Las Vegas gambles with time; Monte Carlo gambles with correctness.* Problem 9 asks you to implement one of each for the same task and contrast their guarantees directly. Note that a Monte Carlo algorithm with a *checkable* answer can often be turned into a Las Vegas one (repeat until the check passes).

2 Probability Review

We work in a finite (or countable) probability space. The only tools we need are the following.

Definition 2.1 (Expectation). *For a discrete random variable X taking values x with probabilities $\Pr[X = x]$, the expectation (mean) is*

$$\mathbb{E}[X] = \sum_x x \cdot \Pr[X = x].$$

Definition 2.2 (Indicator random variable). *For an event A , the indicator $\mathbf{1}_A$ (also written X_A) is 1 if A occurs and 0 otherwise. Its expectation is exactly the probability of the event:*

$$\mathbb{E}[\mathbf{1}_A] = 1 \cdot \Pr[A] + 0 \cdot \Pr[\bar{A}] = \Pr[A].$$

The single most important tool of the week is the following, which holds *whether or not* the variables are independent.

Theorem 2.3 (Linearity of expectation). *For any random variables $X_1, \dots, X_n$ (defined on the same probability space) and constants $a_1, \dots, a_n$,*

$$\mathbb{E} \left[\sum_{i=1}^n a_i X_i \right] = \sum_{i=1}^n a_i \mathbb{E}[X_i].$$

The power of linearity is that it lets us compute the expectation of a complicated quantity (e.g. “number of comparisons made by QuickSort”) by writing it as a *sum of indicators*, computing the easy probability of each indicator, and adding up – never needing to know how the indicators interact. This pattern – **decompose into indicators, then sum** – recurs in nearly every proof below.

Definition 2.4 (Independence; conditional probability). *Events A and B are independent if $\Pr[A \cap B] = \Pr[A] \Pr[B]$. The conditional probability of A given B (with $\Pr[B] > 0$) is $\Pr[A \mid B] = \Pr[A \cap B] / \Pr[B]$. We will repeatedly bound a product of conditional probabilities, as in Karger’s analysis.*

Theorem 2.5 (Union bound). *For any events $A_1, \dots, A_n$ (no independence required),*

$$\Pr \left[\bigcup_{i=1}^n A_i \right] \leq \sum_{i=1}^n \Pr[A_i].$$

The union bound is the workhorse for proving that “nothing bad ever happens”: if each of n bad events is individually unlikely, their union is still unlikely. We use it to bound the probability that *amplified* randomized algorithms ever fail.

3 Contention Resolution: Breaking Symmetry

Our first case study is the cleanest illustration of why randomness helps at all.

Definition 3.1 (Contention resolution). *There are n identical processes $P_1, \dots, P_n$ all wanting to access a single shared resource (e.g. a broadcast channel). In each discrete round, every process may attempt to access the resource. An attempt by P_i succeeds in a round only if P_i is the only process attempting that round; if two or more attempt simultaneously, all of them fail (a “collision”). The processes cannot communicate or coordinate. How can they all eventually succeed?*

If the processes are deterministic and identical they are doomed: they will all do exactly the same thing every round and collide forever. The fix is randomness: each process attempts *independently* with some probability p per round.

Algorithm 1 Randomized Contention Resolution (one process P_i)

```

1: while  $P_i$  has not yet succeeded do
2:   With probability  $p$ , attempt to access the resource this round
3:   (otherwise, stay silent this round)
4: end while
```

Let $A_{i,t}$ be the event “ P_i attempts and succeeds in round t .” Process P_i succeeds in round t exactly when it attempts (probability p) *and* no other process attempts (probability $(1 - p)^{n-1}$):

$$\Pr[A_{i,t}] = p(1 - p)^{n-1}.$$

This is maximized at $p = 1/n$, giving

$$\Pr[A_{i,t}] = \frac{1}{n} \left(1 - \frac{1}{n}\right)^{n-1} \geq \frac{1}{en},$$

using the standard fact $(1 - 1/n)^{n-1} \geq 1/e$. So with $p = 1/n$ each process succeeds in any given round with probability $\Theta(1/n)$.

Theorem 3.2 (All processes succeed in $O(n \log n)$ rounds). *With $p = 1/n$, after $t = \lceil 2en \ln n \rceil$ rounds, all n processes have succeeded at least once with probability at least $1 - 1/n$.*

Proof. Fix a process P_i . By the above, in each round P_i fails to succeed with probability at most $1 - \frac{1}{en}$. Rounds are independent, so the probability P_i has *not* succeeded in any of the first t rounds is at most

$$\left(1 - \frac{1}{en}\right)^t \leq e^{-t/(en)},$$

using $1 - x \leq e^{-x}$. Setting $t = \lceil 2en \ln n \rceil$ gives this at most $e^{-2 \ln n} = 1/n^2$. Now apply the **union bound** (Theorem 2.5) over the n “failure” events: the probability that *some* process has not yet succeeded is at most $n \cdot 1/n^2 = 1/n$. Hence with probability $\geq 1 - 1/n$ all n processes have succeeded within $O(n \log n)$ rounds. $\square$

Problem 5 asks you to simulate this and check empirically that the number of rounds until all succeed grows like $\Theta(n \log n)$.

4 Randomized QuickSort

4.1 The algorithm

QuickSort partitions an array around a *pivot* and recurses. Its worst case is $\Theta(n^2)$ – triggered, for a fixed pivot rule, by adversarial inputs (e.g. an already-sorted array with “first element” as pivot). The randomized version chooses the pivot *uniformly at random*, which makes the expected running time $O(n \log n)$ on *every* input, with no bad inputs at all.

Algorithm 2 Randomized QuickSort

```

1: function RQUICKSORT( $A$ )
2:   if  $|A| \leq 1$  then return  $A$ 
3:   end if
4:   Choose a pivot index uniformly at random; let  $x$  be that element
5:   Partition  $A \setminus \{x\}$  into  $L = \{y < x\}$  and  $R = \{y > x\}$ 
6:   return RQUICKSORT( $L$ ) +  $[x]$  + RQUICKSORT( $R$ )
7: end function

```

4.2 Expected number of comparisons

The running time is dominated by element comparisons, so we count those.

Theorem 4.1. *The expected number of comparisons made by randomized QuickSort on an array of n distinct elements is at most $2n \ln n = O(n \log n)$.*

Proof. Let the sorted order of the elements be $z_1 < z_2 < \dots < z_n$. The whole analysis rests on one clean observation: *any two elements z_i and z_j are compared at most once over the entire execution* (a comparison happens only against a pivot, and once an element is a pivot it is removed from all recursive calls).

For $i < j$, let X_{ij} be the indicator of the event “ z_i and z_j are ever compared.” The total number of comparisons is $X = \sum_{i < j} X_{ij}$, so by **linearity of expectation** (Theorem 2.3),

$$\mathbb{E}[X] = \sum_{i < j} \mathbb{E}[X_{ij}] = \sum_{i < j} \Pr[z_i \text{ and } z_j \text{ are compared}].$$

Key probability. Consider the set $Z_{ij} = \{z_i, z_{i+1}, \dots, z_j\}$ of the $j - i + 1$ elements between z_i and z_j inclusive. Track the first element of Z_{ij} ever chosen as a pivot (every element of Z_{ij} is eventually a pivot or separated from the others only by some element of Z_{ij} being chosen first). Two cases:

- If the first pivot chosen from Z_{ij} is z_i or z_j , then z_i and z_j are compared (the pivot is compared to everyone in its subarray, which still contains the other).
- If the first pivot chosen from Z_{ij} is some z_k with $i < k < j$, then z_i and z_j are split into different subarrays and *never* compared.

Since the pivot is chosen uniformly at random, the first pivot from Z_{ij} is equally likely to be any of its $j - i + 1$ elements. Exactly 2 of those (z_i and z_j) cause a comparison, so

$$\Pr[z_i \text{ and } z_j \text{ compared}] = \frac{2}{j - i + 1}.$$

Summing. Therefore

$$\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j - i + 1} \leq \sum_{i=1}^n \sum_{k=2}^n \frac{2}{k} = 2n \sum_{k=2}^n \frac{1}{k} \leq 2n \ln n,$$

where the last step uses the harmonic bound $\sum_{k=1}^n 1/k \leq 1 + \ln n$. Hence $\mathbb{E}[X] = O(n \log n)$. $\square$

Problem 2 asks you to implement randomized QuickSort and verify both correctness (against `sorted()`) and that the comparison count tracks $O(n \log n)$.

5 Randomized Quickselect

The same idea, restricted to one side of the recursion, finds the k -th smallest element (the k -th *order statistic*) in expected *linear* time.

Theorem 5.1. *Randomized Quickselect runs in expected $O(n)$ time on an array of size n .*

Proof sketch. Call a pivot *good* if it lands in the middle half (between the 25th and 75th percentiles); a uniformly random pivot is good with probability $\geq 1/2$. A good pivot shrinks the surviving subarray to at most $\frac{3}{4}$ of its size. Group the recursion into “phases,” where a phase ends each time a good pivot is hit. The expected number of pivots per phase is at most 2 (a geometric random variable with success probability $\geq 1/2$), and the subarray size shrinks by a factor $\geq \frac{3}{4}$ each phase. So the expected total work is at most

$$\sum_{\text{phases } j \geq 0} 2 \cdot c \left(\frac{3}{4}\right)^j n = 2cn \sum_{j \geq 0} \left(\frac{3}{4}\right)^j = 2cn \cdot 4 = O(n),$$

Algorithm 3 Randomized Quickselect (find the k -th smallest)

```
1: function RSELECT( $A, k$ )
2:   if  $|A| = 1$  then return  $A[0]$ 
3:   end if
4:   Choose a pivot  $x \in A$  uniformly at random
5:   Partition into  $L = \{y < x\}$  and  $R = \{y > x\}$ 
6:   if  $k \leq |L|$  then return RSELECT( $L, k$ )
7:   else if  $k = |L| + 1$  then return  $x$ 
8:   else return RSELECT( $R, k - |L| - 1$ )
9:   end if
10: end function
```

since the work in a subarray of size m (partitioning) is $\Theta(m)$ and the geometric series converges. Hence the expected running time is $O(n)$. $\square$

Problem 3 asks you to implement Quickselect and verify its output against `sorted()[k-1]` over many seeded instances.

6 Karger's Randomized Global Min-Cut

6.1 The problem and the algorithm

Definition 6.1 (Global minimum cut). *Given a connected undirected (multi)graph $G = (V, E)$, a global cut is a partition of V into two non-empty sets $(S, V \setminus S)$; its size is the number of edges with one endpoint on each side. The global min-cut is a cut of minimum size.*

Karger's algorithm is astonishingly simple: repeatedly *contract* a random edge (merging its two endpoints into one "super-vertex," keeping parallel edges but deleting self-loops) until only two super-vertices remain. The edges between those two final super-vertices form a cut; return it.

Algorithm 4 Karger's Contraction Algorithm

```
1: function CONTRACT( $G = (V, E)$ )
2:   while  $|V| > 2$  do
3:     Choose an edge  $e = (u, v) \in E$  uniformly at random
4:     Contract  $e$ : merge  $u$  and  $v$  into one vertex; keep parallel edges; delete resulting self-loops
5:   end while
6:   return the multiset of edges between the two remaining vertices (a cut)
7: end function
```

6.2 Success probability

Theorem 6.2. *A single run of Karger's contraction algorithm returns a fixed global min-cut with probability at least $\frac{1}{\binom{n}{2}} = \frac{2}{n(n-1)}$.*

Proof. Fix one particular global min-cut C^* of size k . The algorithm returns C^* exactly when it *never* contracts any edge of C^* . We bound the probability of this surviving sequence of contractions.

First, note that every vertex must have degree $\geq k$: if some vertex v had degree $< k$, the cut $(\{v\}, V \setminus \{v\})$ would be smaller than k , contradicting minimality. So at any stage with m super-vertices, the total number of edges is $\geq mk/2$ (sum of degrees $\geq mk$, divided by 2).

At the i -th contraction step there are $n - i + 1$ super-vertices, hence at least $(n - i + 1)k/2$ edges, of which exactly k belong to C^* . So the probability of contracting a C^* -edge at this step is at most

$$\frac{k}{(n - i + 1)k/2} = \frac{2}{n - i + 1}.$$

The probability we *avoid* all C^* -edges through all $n - 2$ contractions is, by the chain rule,

$$\Pr[C^* \text{ survives}] \geq \prod_{i=1}^{n-2} \left(1 - \frac{2}{n - i + 1}\right) = \prod_{i=1}^{n-2} \frac{n - i - 1}{n - i + 1}.$$

This telescopes:

$$\frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdot \frac{n-4}{n-2} \cdots \frac{2}{4} \cdot \frac{1}{3} = \frac{2}{n(n-1)} = \frac{1}{\binom{n}{2}}. \quad \square$$

6.3 Amplification

A success probability of $1/\binom{n}{2} \approx 2/n^2$ seems tiny, but it is *polynomially* small, so a polynomial number of independent repetitions makes failure exponentially unlikely.

Corollary 6.3 (Amplification). *Running Karger's algorithm $T = \binom{n}{2} \ln n$ times independently and keeping the smallest cut found returns a global min-cut with probability at least $1 - 1/n$.*

Proof. Each run independently fails to find the fixed min-cut C^* with probability at most $1 - 1/\binom{n}{2}$. The probability all T runs fail is at most

$$\left(1 - \frac{1}{\binom{n}{2}}\right)^T \leq e^{-T/\binom{n}{2}} = e^{-\ln n} = \frac{1}{n},$$

using $1 - x \leq e^{-x}$. So with probability $\geq 1 - 1/n$ at least one run finds C^* , and keeping the best cut returns it. $\square$

Problem 6 implements a single contraction run; Problem 7 adds amplification and checks against a brute-force / max-flow min-cut on small graphs; Problem 11 empirically verifies the $1/\binom{n}{2}$ floor on the single-run success probability.

7 Reservoir Sampling

Often we must sample from a *stream* of unknown length – we see items one at a time and cannot store them all. *Reservoir sampling* maintains a uniform random sample of k items using only $O(k)$ space and a single pass.

Theorem 7.1. *After processing a stream of $n \geq k$ items, every item is in the reservoir R with probability exactly k/n ; equivalently, R is a uniform random k -subset.*

Proof. We show item x_t ends in R with probability k/n , for every t . For $t \leq k$, x_t starts in R (probability 1); for $t > k$, x_t enters R with probability k/t . In either case, after step t the item is in R with probability k/t (for $t \leq k$ this is $1 \geq$, handled directly; we focus on the inductive survival). For each later step $s > t$, x_t *survives* unless step s both replaces (probability k/s) and

Algorithm 5 Reservoir Sampling (sample k of an n -item stream)

```
1:  $R \leftarrow$  the first  $k$  items of the stream
2: for  $t = k + 1, k + 2, \dots$  (each new item  $x_t$ ) do
3:   With probability  $k/t$ , replace a uniformly random element of  $R$  with  $x_t$ 
4: end for
5: return  $R$ 
```

selects x_t 's slot (probability $1/k$), i.e. it is removed with probability $1/s$, surviving with probability $1 - 1/s = (s - 1)/s$. Hence

$$\Pr[x_t \in R \text{ at the end}] = \frac{k}{t} \prod_{s=t+1}^n \frac{s-1}{s} = \frac{k}{t} \cdot \frac{t}{n} = \frac{k}{n},$$

the product telescoping. (For $t \leq k$ the leading factor is 1 and the same telescoping gives k/n .) Since every item is included with the same probability k/n , R is a uniform random k -subset. $\square$

Problem 4 asks you to implement reservoir sampling and verify the k/n inclusion frequency empirically over many seeded trials.

8 Hashing and Number-Theoretic Randomized Algorithms

8.1 Universal hashing

A *hash function* maps keys to a small range $\{0, \dots, m - 1\}$ of buckets. A fixed hash function can always be defeated by an adversary who piles all keys into one bucket. *Universal hashing* defeats the adversary by choosing the hash function *at random* from a carefully designed family.

Definition 8.1 (Universal family). A family $\mathcal{H}$ of hash functions $h : U \rightarrow \{0, \dots, m - 1\}$ is universal if for every pair of distinct keys $x \neq y$,

$$\Pr_{h \in \mathcal{H}}[h(x) = h(y)] \leq \frac{1}{m}.$$

A standard universal family: pick a prime $p > |U|$, choose $a \in \{1, \dots, p-1\}$ and $b \in \{0, \dots, p-1\}$ uniformly at random, and set

$$h_{a,b}(x) = ((ax + b) \bmod p) \bmod m.$$

Using indicator variables and linearity of expectation, the *expected* number of colliding pairs when hashing n keys into m buckets is at most

$$\binom{n}{2} \cdot \frac{1}{m} \approx \frac{n^2}{2m},$$

since each of the $\binom{n}{2}$ pairs collides with probability $\leq 1/m$. Problem 12 asks you to implement this $(ax + b \bmod p) \bmod m$ family and verify the expected collision count empirically.

8.2 Monte Carlo estimation of π

A toy but vivid Monte Carlo algorithm: throw N random darts uniformly into the unit square $[0, 1]^2$ and count the fraction landing inside the quarter-circle of radius 1. That fraction approaches the area $\pi/4$, so $4 \times$ (fraction) estimates π . The error shrinks like $1/\sqrt{N}$ (by the variance of a sum of N i.i.d. indicators). Problem 8 implements this and checks the estimate is within a loose tolerance for large N .

8.3 Miller–Rabin primality testing

Deciding whether a large integer is prime by trial division costs $\Theta(\sqrt{n})$ – hopeless for cryptographic-size numbers. The **Miller–Rabin** test is a Monte Carlo algorithm that, for a candidate n , picks random “witnesses” a and checks an algebraic condition (derived from Fermat’s little theorem, refined to catch Carmichael numbers). If n is prime, the test *always* reports “probably prime.” If n is composite, each random witness exposes it with probability $\geq 3/4$, so r independent witnesses reduce the error to $\leq (1/4)^r$ – amplification again. Problem 10 implements Miller–Rabin and checks it against deterministic trial division over a range.

Algorithm 6 Miller–Rabin (one witness a for odd $n > 2$)

```
1: Write  $n - 1 = 2^s d$  with  $d$  odd
2:  $x \leftarrow a^d \bmod n$ 
3: if  $x = 1$  or  $x = n - 1$  then return “probably prime”
4: end if
5: for  $r = 1$  to  $s - 1$  do
6:    $x \leftarrow x^2 \bmod n$ 
7:   if  $x = n - 1$  then return “probably prime”
8:   end if
9: end for
10: return “composite”
```

9 Summary and Course Wrap-Up

9.1 This week

Randomization gives us a third design paradigm, alongside the deterministic techniques (greedy, divide-and-conquer, dynamic programming, network flow) of earlier weeks. The recurring analytical patterns were:

- **Indicator variables + linearity of expectation:** count a complicated quantity by decomposing it into a sum of simple 0/1 events and summing their probabilities – used for QuickSort’s comparisons (Theorem 4.1), reservoir sampling, and universal-hashing collisions.
- **The union bound:** prove “nothing bad ever happens” by summing the small probabilities of individual bad events – used for contention resolution (Theorem 3.2) and Karger amplification (Corollary 6.3).
- **Amplification:** turn a small-but-polynomial success probability into an overwhelming one by independent repetition – Karger’s min-cut and Miller–Rabin.
- **Las Vegas vs. Monte Carlo:** gamble with time (always correct) or with correctness (always fast), and convert between them when the answer is checkable.

9.2 Looking back over the course

This is the final week of TOAE209/TOAH209. Over fifteen weeks we built a toolkit for *designing* and *proving correct* algorithms:

- **Foundations** (Weeks 1–3): stable matching, asymptotic analysis, graphs and union-find.
- **Greedy** (Weeks 4–5): exchange arguments and “greedy stays ahead,” applied to scheduling, shortest paths, and minimum spanning trees.
- **Divide and conquer** (Weeks 6–7): recurrences, the Master Theorem, mergesort, fast multiplication, and the FFT.
- **Dynamic programming** (Weeks 8–9): optimal substructure, memoization, and a gallery of DP formulations.
- **Network flow** (Weeks 10–11): max-flow/min-cut and its many reductions (bipartite matching, project selection, image segmentation).
- **Complexity and coping with hardness** (Weeks 12–14): NP-completeness, PSPACE, parameterized/structural tractability, approximation algorithms, and local search.
- **Randomization** (Week 15): a new paradigm that trades certainty for simplicity, speed, and the ability to break symmetry.

The unifying lesson is that an algorithm is not finished when it runs – it is finished when you can *prove* what it does and how fast. Whether the proof is an exchange argument, an induction on subproblems, a flow-conservation argument, an NP-completeness reduction, or an expectation computed with indicator variables, the discipline of pairing every algorithm with its proof is what distinguishes algorithm design from mere programming. Carry that discipline forward.